

Brain Tumor MRI Classification Using Deep Learning and Transfer Learning

A Deep Learning Approach Using ResNet50 Transfer Learning for Four-Class Brain Tumor MRI Image Classification

Domain	Computer Vision, Medical Image Classification, Deep Learning
Framework	PyTorch / Torchvision
Model	ResNet50 (Transfer Learning)
Development Environment	Google Colab
Dataset Source	Kaggle - Brain Tumor MRI Dataset (Masoud Nickparvar)

Abstract

This project presents a deep learning system for the classification of brain tumors from Magnetic Resonance Imaging (MRI) scans into four categories: Glioma, Meningioma, Pituitary, and No Tumor. The work uses the publicly available Brain Tumor MRI Dataset on Kaggle, curated by Masoud Nickparvar, and applies transfer learning using a ResNet50 convolutional neural network pretrained on ImageNet. The complete workflow, developed and executed in Google Colab, includes dataset acquisition through KaggleHub, dataset loading with torchvision's ImageFolder, image preprocessing, augmentation, an 80/20 training-validation split, and fine-tuning of the deeper layers of ResNet50 while keeping earlier layers frozen. The model was trained for 15 epochs using the Adam optimizer and cross-entropy loss, achieving a best validation accuracy of 95.00%. On the held-out test set, the model achieved a test accuracy of 90.69%, a multiclass ROC-AUC of 0.9821, and a macro PR-AUC of 0.9523, indicating strong overall class-separation and precision-recall performance. In addition to model development, this report outlines the design of a planned interactive frontend to be built with IPyWidgets inside Google Colab, allowing a user to upload an MRI image and receive a predicted class along with confidence scores. This project is developed strictly for educational and research purposes as part of a BS Data Science final-year project. It is an experimental prototype and is not intended to replace a qualified radiologist, provide a clinical diagnosis, or be used in any real-world clinical setting without further validation and expert medical review.

Keywords: Brain Tumor Classification, MRI, Deep Learning, Convolutional Neural Networks, ResNet50, Transfer Learning, PyTorch, Medical Image Analysis

Table of Contents

1. Introduction
2. Background and Motivation
3. Problem Statement
4. Project Objectives
5. Dataset Description
6. Tools and Technologies
7. Methodology / Project Workflow
8. Data Preprocessing
9. Model Architecture
10. Transfer Learning and Fine-Tuning
11. Training Procedure
12. Experimental Results
13. Model Evaluation
14. Google Colab Frontend Plan
15. Limitations
16. Future Work
17. Completed Work vs Planned Work
18. Conclusion
19. References
20. Appendix

1. Introduction

Brain tumor detection and classification from MRI images is an important application of artificial intelligence and medical computer vision. Manual diagnosis of brain tumors from MRI scans is a time-consuming task that requires significant expertise from radiologists. Deep learning offers the ability to learn visual patterns directly from MRI scans and classify them into different tumor categories, potentially assisting in faster preliminary screening.

This project is an educational and research-based prototype. It is **not intended to replace a qualified radiologist or provide a clinical diagnosis**. All results and conclusions in this report should be interpreted strictly within the context of an academic learning exercise.

2. Background and Motivation

Deep learning models, particularly Convolutional Neural Networks (CNNs), have shown strong capability in learning hierarchical visual features from images, making them well suited to medical image analysis tasks such as MRI-based tumor classification. This project was undertaken with the following motivations:

- Learning practical computer vision techniques
- Applying deep learning to medical images
- Understanding transfer learning as a training strategy
- Working with a real-world image dataset
- Evaluating a multiclass classification model
- Creating a simple frontend where users can upload an MRI image
- Improving the author's machine learning portfolio

3. Problem Statement

Given an MRI brain scan image, the goal is to automatically classify the image into one of four categories: Glioma, Meningioma, Pituitary, or No Tumor. This is framed as a supervised, single-label, four-class image classification problem, not a tumor segmentation problem. The model predicts the category of an entire image rather than localizing or outlining a tumor region within the image.

4. Project Objectives

- 1 Download and prepare a brain tumor MRI image dataset.
- 2 Explore the dataset structure and class distribution.
- 3 Preprocess MRI images for deep learning.
- 4 Apply image augmentation to improve model generalization.
- 5 Use a pretrained ResNet50 model for transfer learning.
- 6 Fine-tune selected layers of the model.
- 7 Train the model for multiclass MRI classification.

- 8 Evaluate the model using accuracy, loss, ROC-AUC, PR-AUC, classification report, and confusion matrix.
- 9 Save the best-performing model checkpoint.
- 10 Develop a Google Colab-based frontend where users can upload an MRI image and receive a predicted class with confidence scores.
- 11 Document the complete workflow for future improvements.

5. Dataset Description

The project uses the publicly available **Brain Tumor MRI Dataset** from Kaggle, created by **Masoud Nickparvar**, available at: <https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>.

The dataset contains MRI images organized into two main folders, **Training** and **Testing**, with images further divided into four class subfolders. The four classes are:

- **Glioma** - A tumor originating from glial cells or related brain-supporting tissue.
- **Meningioma** - A tumor that develops from the meninges, the protective membranes surrounding the brain and spinal cord.
- **Pituitary** - A tumor associated with the pituitary gland.
- **No Tumor** - MRI images without a visible tumor according to the dataset labels.

This project treats the task as a **four-class image classification problem**. It is important to note that the dataset is an image classification dataset, not a tumor segmentation dataset: the model predicts the category of an image rather than drawing the exact tumor boundary.

Dataset Summary

Dataset Component	Description
Dataset source	Kaggle - Brain Tumor MRI Dataset (Masoud Nickparvar)
Dataset format	MRI images organized in class-labeled folders
Main folders	Training, Testing
Number of classes	4
Class names	Glioma, Meningioma, Pituitary, No Tumor
Task type	Multiclass image classification (not segmentation)
Image type	Brain MRI scans
Total number of images	To be added (verified from actual loaded dataset output)

6. Tools and Technologies

The following table summarizes the technologies used in this project and their roles.

Tool / Library	Role in the Project
Google Colab	Cloud-based notebook environment used for model development and training, with c
Python	Primary programming language used for the entire pipeline.
PyTorch	Deep learning framework used to build, train, and evaluate the ResNet50 model.
Torchvision	Provides pretrained models (ResNet50), datasets utilities (ImageFolder), and image

Tool / Library	Role in the Project
KaggleHub	Used to download the Brain Tumor MRI Dataset directly inside Google Colab.
NumPy	Used for numerical operations on arrays and data structures.
Matplotlib	Used for plotting training curves and other visualizations.
PIL / Pillow	Used for image loading and manipulation, including in the planned frontend.
Scikit-learn	Used for evaluation metrics such as ROC-AUC, PR-AUC, and classification reports.
IPyWidgets	Used to build the interactive Google Colab frontend (upload button, output display).
CUDA / GPU (when available)	Used to accelerate model training when a GPU runtime is available in Colab.

Google Colab was used for model development and training. GPU acceleration, when available, can significantly reduce training time compared with CPU execution; however, no specific GPU model is claimed here as this information was not explicitly provided.

7. Methodology - Project Workflow

The complete pipeline followed in this project is summarized below:

Dataset Download → Dataset Inspection → Image Preprocessing → Augmentation → Train/Validation Split → DataLoader → ResNet50 Transfer Learning → Fine-Tuning → Training → Best Checkpoint Saving → Testing → Evaluation → Google Colab Frontend

7.1 Dataset Download and Loading

The dataset was downloaded in Google Colab using KaggleHub, a library that simplifies downloading Kaggle datasets directly into the Colab environment.

```
!pip install -q kagglehub

import kagglehub
import os

path = kagglehub.dataset_download(
    "masoudnickparvar/brain-tumor-mri-dataset"
)

print("Path to dataset files:", path)
print("Files and folders:", os.listdir(path))
```

After downloading, the training and testing folders were assigned to variables:

```
TRAIN_DIR = os.path.join(path, "Training")
TEST_DIR = os.path.join(path, "Testing")
```

The folder structure was inspected using Python's `os.walk()` function, which recursively lists all subdirectories and files, allowing verification that the Training and Testing folders contained the expected four class subfolders.

`torchvision.datasets.ImageFolder` was then used to automatically load images and assign class labels based on folder names, without requiring a separate manual labeling step.

```
from torchvision import datasets

full_train_dataset = datasets.ImageFolder(TRAIN_DIR)
test_dataset = datasets.ImageFolder(TEST_DIR)

print("Classes:", full_train_dataset.classes)
print("Class mapping:", full_train_dataset.class_to_idx)
print("Training images:", len(full_train_dataset))
print("Testing images:", len(test_dataset))
```

ImageFolder is a torchvision utility that assumes each subfolder of the root directory represents a distinct class. The `classes` attribute returns the list of class names discovered from the folder structure, and `class_to_idx` returns a dictionary mapping each class name to an integer label (for example, `{'glioma': 0, 'meningioma': 1, 'notumor': 2, 'pituitary': 3}`). This is how folder names are automatically converted into numerical labels used during training.

8. Data Preprocessing

All MRI images were resized to a fixed size of **224 x 224 pixels**. A fixed image size is required because convolutional neural networks, and specifically the pretrained ResNet50 architecture used in this project, expect a consistent input tensor shape for every image passed through the network. The value 224 x 224 also matches the input size that ResNet50 was originally trained on using the ImageNet dataset.

8.1 Training Transformations

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(
        brightness=0.15,
        contrast=0.15
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

- **Resize** - scales every image to 224 x 224 pixels so that all inputs share the same dimensions.
- **Random horizontal flip** - randomly mirrors the image with a probability of 0.5, helping the model become invariant to left-right orientation.
- **Random rotation** - randomly rotates the image by up to 10 degrees, simulating slight variations in scan alignment.
- **Brightness and contrast adjustment (ColorJitter)** - randomly varies brightness and contrast by up to 15%, helping the model generalize to differences in scanner or imaging settings.
- **Conversion to tensor** - converts the image from a PIL image into a PyTorch tensor so it can be processed by the neural network.
- **Normalization** - rescales pixel values using fixed mean and standard deviation values so that the input distribution matches what the pretrained ResNet50 model expects.

8.2 Validation / Testing Transformations

```
val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Random augmentation is applied only to training images so that the model sees varied versions of the data during learning, which helps prevent overfitting. Validation and testing images are not augmented, because the goal during evaluation is to measure performance on realistic, unmodified images that represent how the model would encounter data in practice.

The normalization mean and standard deviation values used ([0.485, 0.456, 0.406] and [0.229, 0.224, 0.225]) are the standard values commonly used with ImageNet-pretrained models, since ResNet50 was originally trained on images normalized with these statistics.

9. Training and Validation Split

The original training dataset was divided into **80% training data** and **20% validation data**. A validation set is necessary because it allows the model's performance to be measured on data that was not used to update the model's weights, giving a more realistic estimate of how well the model generalizes and helping to detect overfitting during training. A fixed random seed of **42** was used for the split to make the results reproducible.

Set	Purpose
Training set	Used to update model weights through backpropagation during training.
Validation set	Used to monitor performance on unseen data during training and to decide when to save
Testing set	Used only after training is complete, to provide a final, unbiased evaluation of the model.

10. DataLoaders

PyTorch DataLoaders are used to efficiently feed batches of images and labels into the model during training and evaluation. The project used a batch size of:

```
BATCH_SIZE = 32
```

- **Batch size** - the number of images processed together in a single forward and backward pass; a batch size of 32 balances memory usage and training stability.
- **Shuffling** - training data is shuffled at every epoch so the model does not learn any unintended order-based patterns in the data.
- **Validation and test data are not shuffled**, since order does not affect evaluation and keeping a fixed order makes results easier to inspect and reproduce.
- **Parallel data loading** - multiple worker processes can load and preprocess images in parallel while the GPU or CPU trains on the current batch, reducing idle time.
- **num_workers** - controls how many subprocesses are used for data loading in parallel.
- **pin_memory** - when using a GPU, this setting can speed up the transfer of data from CPU memory to GPU memory.

11. Model Architecture - ResNet50

The main deep learning model used in this project is **ResNet50**, a well-known convolutional neural network architecture.

11.1 Convolutional Neural Networks (CNNs)

A Convolutional Neural Network (CNN) is a type of neural network designed specifically to process grid-like data such as images. CNNs use convolutional layers that scan small regions of the image with learnable filters, allowing the network to automatically detect visual patterns such as edges, textures, and shapes, and to combine these into increasingly complex features in deeper layers.

11.2 Transfer Learning and Pretrained Models

Transfer learning is a technique where a model that has already been trained on a large dataset (such as ImageNet, which contains millions of general images) is reused as a starting point for a new, related task. A pretrained model already contains useful visual features, such as edge and texture detectors, that were learned from a large and diverse dataset. Reusing these features instead of training a network from scratch can significantly reduce the amount of data and training time needed to achieve good performance on a new task, such as MRI classification.

11.3 Why ResNet50 Was Selected

ResNet50 was selected because it is a widely used, well-validated architecture that is known to perform well on image classification tasks and is readily available as a pretrained model in torchvision. Its residual learning design also makes it easier to fine-tune for new tasks such as MRI classification.

11.4 Residual Connections and Skip Connections

One of the key innovations of ResNet50 is the use of **residual connections**, also called **skip connections**. In a standard deep network, each layer's output is passed directly to the next layer. In ResNet, a shortcut connection allows the input to a block of layers to be added directly to the output of that block. This helps address the **vanishing-gradient problem**, where gradients become extremely small as they are propagated backward through many layers during training, making it difficult for earlier layers to learn effectively. Skip connections allow gradients to flow more directly through the network, which makes it possible to train much deeper networks successfully. As a result, deeper networks like ResNet50 can benefit from residual learning by continuing to improve performance as more layers are added, rather than suffering from degraded training as network depth increases.

11.5 General Structure of ResNet50

ResNet50 consists of an initial convolutional layer, followed by four main stages of residual blocks (commonly referred to as layer1 through layer4), and finally a global pooling operation followed by a fully connected classification layer. The convolutional layers throughout the network are responsible for feature extraction: earlier layers detect simple, general visual patterns such as edges and colors, while deeper layers combine these into more complex, task-specific patterns. The final fully connected layer takes the extracted features and maps them to output class scores.

In this project, the model was initialized with ImageNet pretrained weights, meaning the convolutional layers already contained useful, general-purpose visual features before any MRI-specific training took place.

```
weights = models.ResNet50_Weights.DEFAULT  
model = models.resnet50(weights=weights)
```

12. Transfer Learning and Fine-Tuning Strategy

Most of the pretrained ResNet50 layers were initially frozen, meaning their weights were not updated during training. This preserves the general-purpose visual features already learned from ImageNet.

```
for parameter in model.parameters():
    parameter.requires_grad = False
```

The original final classification layer, which was designed for 1000 ImageNet classes, was replaced with a new fully connected layer suitable for the four MRI classes used in this project:

```
number_of_classes = len(full_train_dataset.classes)

model.fc = nn.Linear(
    model.fc.in_features,
    number_of_classes
)
```

This new final classification layer learns to distinguish between the four target classes: Glioma, Meningioma, Pituitary, and No Tumor.

In addition to replacing the final layer, the Layer4 section of ResNet50, its deepest set of residual blocks, was unfrozen for fine-tuning:

```
for parameter in model.layer4.parameters():
    parameter.requires_grad = True
```

Fine-tuning only selected deeper layers, rather than the entire network, can be useful because:

- Earlier layers learn general visual patterns that are still useful for MRI images.
- Deeper layers learn more task-specific patterns that benefit from adaptation to the new domain.
- It reduces training cost, since fewer parameters need to be updated.
- It can help prevent overfitting, especially with a moderately sized dataset.
- It adapts pretrained features to MRI images without discarding useful general knowledge.

Finally, the model was moved to the available computation device (CPU or GPU):

```
model = model.to(device)
```

13. Loss Function and Optimizer

The project used cross-entropy loss as the objective function:

```
criterion = nn.CrossEntropyLoss()
```

Cross-entropy loss is appropriate for multiclass classification because it measures the difference between the predicted probability distribution over classes and the true class label, penalizing the model more heavily when it assigns low probability to the correct class.

The optimizer used was Adam:

```
optimizer = optim.Adam(
    filter(
```

```
        lambda parameter: parameter.requires_grad,
        model.parameters()
    ),
    lr=1e-5
)
```

- **Adam optimizer** - an adaptive optimization algorithm that adjusts the learning rate for each parameter individually, generally leading to stable and efficient training.
- **Learning rate** - controls the size of the steps taken when updating model weights.
- **Why a small learning rate (1e-5) was used** - since the model is being fine-tuned from pretrained weights rather than trained from scratch, a small learning rate helps avoid destroying the useful pretrained features through overly large weight updates.
- **Why only trainable parameters were passed to the optimizer** - the filter ensures that only parameters with `requires_grad` set to `True` (the new classification layer and layer4) are updated, consistent with the freezing strategy described above.

14. Training Procedure

The model was trained for **15 epochs**. Each epoch followed this general procedure:

- 1 Set the model to training mode.
- 2 Load a batch of MRI images.
- 3 Move images and labels to the selected device.
- 4 Clear previous gradients.
- 5 Perform forward propagation.
- 6 Calculate the loss.
- 7 Perform backpropagation.
- 8 Update trainable model parameters.
- 9 Calculate training loss and accuracy.
- 10 Evaluate the model on the validation set.
- 11 Save the model if validation accuracy improves.

`model.train()` switches the model into training mode, which enables layers such as dropout and batch normalization to behave appropriately for training (for example, batch normalization uses batch statistics). `model.eval()` switches the model into evaluation mode, so that these layers instead use fixed, learned statistics, which is appropriate for validation and testing.

`torch.no_grad()` is used during validation and testing to disable gradient computation. Since no weight updates occur during evaluation, disabling gradient tracking reduces memory usage and speeds up computation.

15. Best Checkpoint Strategy

The best model checkpoint was saved whenever validation accuracy improved compared to the previous best value. The best validation accuracy achieved during training was:

95.00%

The best checkpoint was saved using a structure similar to:

```
torch.save({
    "model_state_dict": model.state_dict(),
    "class_names": full_train_dataset.classes,
    "best_val_accuracy": best_val_accuracy
}, "/content/best_resnet50_brain_tumor.pth")
```

Saving the best checkpoint, rather than automatically using the weights from the final epoch, is important because validation performance can fluctuate across epochs. The final epoch is not always the epoch with the best validation accuracy, so relying on the best-saved checkpoint ensures that the strongest-performing version of the model on unseen validation data is retained for evaluation and deployment.

16. Complete Training Results

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Checkpoint
1	1.2285	0.5971	1.0524	0.8214	Yes
2	0.7306	0.8359	0.6424	0.8634	Yes
3	0.4409	0.8672	0.4770	0.8795	Yes
4	0.3393	0.8868	0.4544	0.8875	Yes
5	0.2783	0.9065	0.2885	0.9089	Yes
6	0.2504	0.9167	0.4297	0.9071	No
7	0.2138	0.9261	0.3442	0.9161	Yes
8	0.1866	0.9355	0.2168	0.9304	Yes
9	0.1795	0.9373	0.5951	0.9277	No
10	0.1587	0.9462	0.2641	0.9330	Yes
11	0.1424	0.9520	0.2530	0.9420	Yes
12	0.1278	0.9583	0.2041	0.9402	No
13	0.1145	0.9618	0.1815	0.9437	Yes
14	0.1107	0.9665	0.1792	0.9473	Yes
15	0.0969	0.9679	0.2767	0.9500	Yes

Main observations:

- Training accuracy increased from 59.71% to 96.79%.
- Validation accuracy increased from 82.14% to 95.00%.
- Training loss generally decreased across epochs.
- Validation accuracy improved gradually over the course of training.
- Some fluctuations appeared in validation loss and accuracy, notably at epochs 6, 9, and 12, where the checkpoint was not updated.
- The best validation accuracy was achieved at epoch 15.
- The difference between training and validation performance should be monitored for possible overfitting, although no severe overfitting is asserted here without further analysis.

Training and Validation Curves

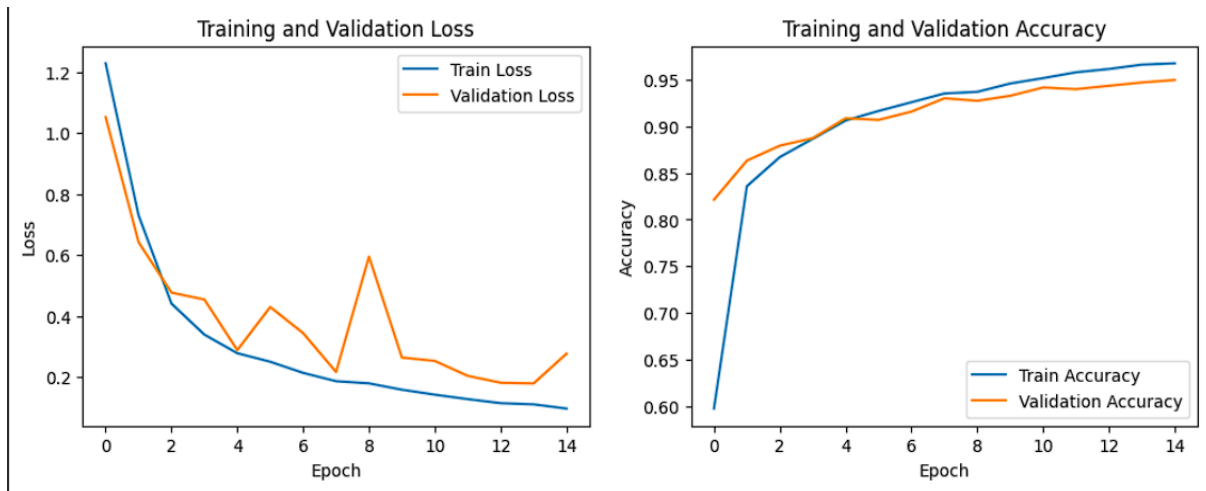


Figure 1: Training and validation loss (left) and training and validation accuracy (right) across 15 epochs.

The loss curve shows that training loss decreased steadily throughout training, while validation loss decreased overall but showed some fluctuation, including a noticeable spike around epoch 8. The accuracy curve shows that training accuracy increased consistently, while validation accuracy also increased overall, tracking closely with training accuracy for much of training before a small gap emerges in later epochs. This type of curve is useful for visually identifying the point at which the model begins to overfit, if the training and validation curves start to diverge significantly.

17. Final Evaluation Results

Metric	Result
Test Loss	0.5530
Test Accuracy	90.69%
Best Validation Accuracy	95.00%
Multiclass ROC-AUC	0.9821
Macro PR-AUC	0.9523
Final Training Accuracy	96.79%
Final Validation Accuracy	95.00%

Test Loss

Test loss measures how far the model's predicted probability distribution is from the true labels on the held-out test set. Lower values indicate predictions that are closer to the correct labels.

Test Accuracy

Test accuracy represents the proportion of test images that were classified correctly. A test accuracy of 90.69% means the model correctly classified approximately 9 out of every 10 test images.

Validation Accuracy

Validation accuracy measures performance on the held-out validation subset during training, and was used to select the best model checkpoint.

Multiclass ROC-AUC

ROC-AUC evaluates how well the model separates different classes across classification thresholds. A multiclass ROC-AUC of 0.9821 indicates strong class-separation performance on the evaluated data.

Macro PR-AUC

Macro PR-AUC summarizes precision-recall performance across the four classes by giving each class equal importance, regardless of how many samples each class contains. A macro PR-AUC of 0.9523 indicates strong precision-recall performance overall.

These metrics should not be interpreted as proof of clinical reliability. They reflect performance on a specific public dataset under experimental conditions only.

18. Model Evaluation Visualizations

The following visualizations are included, or noted as placeholders where not yet generated, to support model evaluation:

Training and validation loss curve

What it shows: Shows how loss changes across epochs for both training and validation sets.

Why it is useful: Useful for identifying whether the model is learning effectively and whether overfitting is occurring.

How to interpret it: A widening gap between training and validation loss over time can indicate overfitting.

Training and validation accuracy curve

What it shows: Shows how classification accuracy changes across epochs.

Why it is useful: Useful for tracking overall learning progress and comparing training versus validation performance.

How to interpret it: Accuracy curves that plateau or diverge can indicate the model has reached its learning capacity or is starting to overfit.

Confusion matrix

What it shows: Shows the number of correct and incorrect predictions for each class, arranged as true class versus predicted class.

Why it is useful: Useful for identifying which classes are most often confused with one another.

How to interpret it: Diagonal values represent correct predictions; off-diagonal values represent misclassifications between specific class pairs.

Classification report

What it shows: Reports precision, recall, and F1-score for each class.

Why it is useful: Useful for understanding per-class performance beyond overall accuracy.

How to interpret it: Low precision or recall for a specific class indicates weaker performance for that category.

[Insert classification report here]

Multiclass ROC curve

What it shows: Plots true positive rate against false positive rate for each class.

Why it is useful: Useful for visualizing class-separation performance across thresholds.

How to interpret it: Curves closer to the top-left corner indicate stronger performance.

[Insert multiclass roc curve here]

Precision-recall curve

What it shows: Plots precision against recall for each class.

Why it is useful: Especially useful when classes are imbalanced.

How to interpret it: Curves closer to the top-right corner indicate stronger performance.

[Insert precision-recall curve here]

Example MRI predictions

What it shows: Shows sample MRI images alongside their predicted and true labels.

Why it is useful: Useful for qualitative inspection of model behavior.

How to interpret it: Correct and incorrect examples both provide insight into model strengths and weaknesses.

[Insert example mri predictions here]

Class distribution chart

What it shows: Shows the number of images per class in the dataset.

Why it is useful: Useful for identifying class imbalance that could affect training or evaluation.

How to interpret it: A skewed distribution may require additional strategies such as weighted loss.

[Insert class distribution chart here]

19. Classification Report and Confusion Matrix

Precision, recall, F1-score, and support are standard classification metrics that provide more detail than overall accuracy alone:

- **Precision** - of all images predicted as a given class, the proportion that actually belong to that class.
- **Recall** - of all images that actually belong to a given class, the proportion correctly identified by the model.
- **F1-score** - the harmonic mean of precision and recall, providing a single balanced metric.
- **Support** - the number of true instances of each class in the evaluated dataset.
- **Confusion matrix** - a table showing counts of predictions for each true class versus each predicted class.

These metrics are especially important for medical image classification because overall accuracy alone may hide poor performance on a specific tumor category. For example, a model could achieve high overall accuracy while performing poorly on a less common or harder-to-distinguish class, which is a critical concern in a medical context.

[Insert actual classification report here]

Confusion Matrix

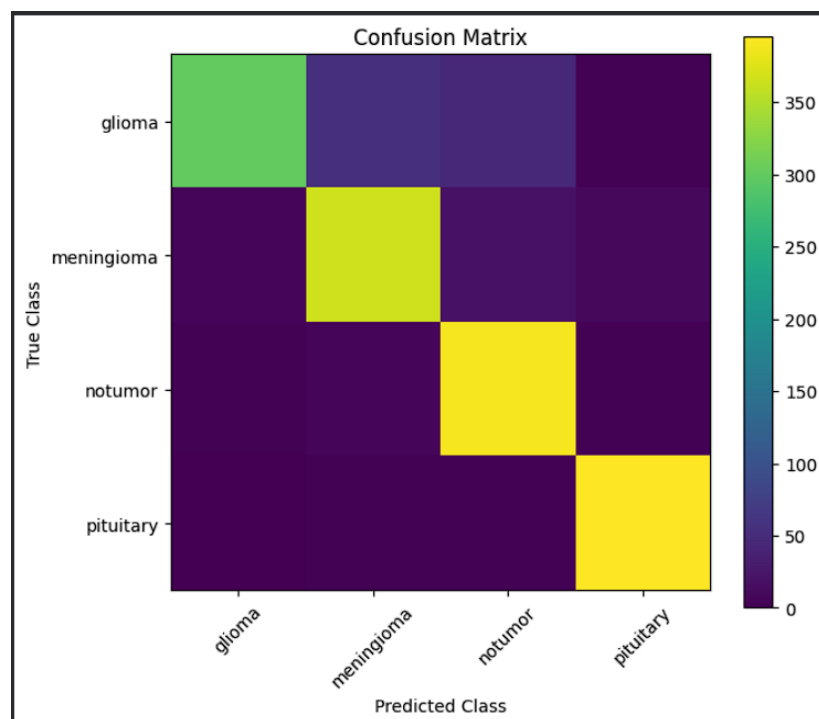


Figure 2: Confusion matrix showing true class versus predicted class counts on the test set.

The confusion matrix shows strong diagonal dominance, indicating that most MRI images were correctly classified into their true category. Some off-diagonal values are visible for the glioma class, suggesting a degree of confusion with other classes; exact class-wise precision, recall, and F1-score values are not stated here since they were not explicitly provided, and should be added from the actual classification

report output.

20. Google Colab Frontend Plan

The next stage of the project is to create a simple interactive frontend directly inside Google Colab, using the following components:

- IPyWidgets
- FileUpload widget
- Button widget
- Output widget
- PIL / Pillow for image handling
- The trained ResNet50 model checkpoint

Expected Workflow

- 1 The user opens the Google Colab notebook.
- 2 The trained model checkpoint is loaded.
- 3 The user uploads an MRI image.
- 4 The image is converted to RGB.
- 5 The same validation/test preprocessing is applied.
- 6 The image is passed to the trained model.
- 7 The model generates class probabilities.
- 8 The class with the highest probability is selected.
- 9 The frontend displays the uploaded MRI image, predicted class, confidence score, probability of each class, and a medical disclaimer.

The frontend will support all four classes: Glioma, Meningioma, Pituitary, and No Tumor. It is important to note that this frontend is a **demonstration interface** and **not a clinical diagnostic tool**.

Planned Frontend Features

- MRI image upload button
- Analyze MRI button
- Clear / reset button
- Display of uploaded image
- Predicted category
- Confidence percentage
- Class probability scores
- Error message when no image is uploaded
- Clear medical disclaimer
- Simple and beginner-friendly interface

The frontend will initially run inside Google Colab using IPyWidgets. A future version could be developed using Streamlit, Gradio, Flask, or FastAPI for a more polished, standalone deployment.

21. Limitations

- The model was trained and evaluated on a specific public dataset.
- Public datasets may not represent all hospitals, scanners, populations, or MRI protocols.
- The test accuracy is lower than the best validation accuracy.
- The model may make incorrect predictions.
- The model does not segment the tumor region.
- The model does not provide a medical explanation by default.
- High ROC-AUC does not guarantee clinical usefulness.
- The model has not been clinically validated.
- The frontend is intended for demonstration and educational purposes.
- Dataset labels and image quality may contain limitations.
- External validation is still required.

22. Future Work

Data Improvements

- Verify class balance
- Add more diverse MRI images
- Use additional publicly available datasets
- Investigate image quality differences
- Check for duplicate or near-duplicate images
- Improve data documentation
- Perform stronger external validation

Model Improvements

- Compare ResNet50 with ResNet18, DenseNet121, EfficientNet, and Vision Transformers
- Experiment with unfreezing more layers
- Use learning-rate scheduling
- Apply early stopping
- Use weighted loss if class imbalance exists
- Try focal loss if appropriate
- Perform systematic hyperparameter tuning
- Compare different image sizes
- Experiment with stronger but medically appropriate augmentation

Evaluation Improvements

- Report class-wise precision, recall, and F1-score

- Analyze the confusion matrix in depth
- Calculate confidence calibration
- Test on an independent dataset
- Perform error analysis
- Inspect incorrectly classified images
- Evaluate robustness to image quality changes
- Use explainability methods such as Grad-CAM

Frontend Improvements

- Build a polished Streamlit or Gradio interface
- Add drag-and-drop upload
- Add prediction history
- Display probability charts
- Add Grad-CAM heatmaps
- Add model information
- Add an exportable prediction report
- Add input validation
- Add a clear disclaimer
- Deploy the application only after proper testing and ethical review

Medical and Ethical Improvements

- Clearly communicate that the system is experimental
- Avoid presenting predictions as confirmed diagnoses
- Protect patient privacy
- Avoid uploading identifiable patient information
- Investigate dataset bias
- Evaluate performance across different scanners, hospitals, and populations
- Seek expert medical review before any real-world clinical use

23. Completed Work vs Planned Work

Status	Item
Completed	Dataset download and loading via KaggleHub and ImageFolder
Completed	Dataset structure exploration
Completed	Image preprocessing and augmentation pipeline
Completed	Training / validation split (80/20, seed = 42)
Completed	DataLoader setup
Completed	ResNet50 transfer learning setup (frozen base, layer4 unfrozen)
Completed	Model training for 15 epochs
Completed	Best checkpoint saving strategy
Completed	Final evaluation (test accuracy, ROC-AUC, PR-AUC)
Completed	Training/validation loss and accuracy curves
Completed	Confusion matrix generation
Planned	Detailed classification report (class-wise precision, recall, F1-score)
Planned	Multiclass ROC and precision-recall curve plots
Planned	Example MRI prediction visualizations
Planned	Class distribution chart
Planned	Interactive Google Colab frontend using IPyWidgets
Planned	Future model comparisons and hyperparameter tuning
Planned	Grad-CAM explainability and expert medical review

24. Conclusion

This project successfully demonstrated a complete deep learning workflow for four-class brain tumor MRI image classification. The workflow included dataset acquisition using KaggleHub, dataset loading using ImageFolder, image preprocessing, data augmentation, training-validation splitting, ResNet50 transfer learning, fine-tuning of deeper layers, model checkpointing, performance evaluation, and ROC-AUC and PR-AUC analysis, along with planning for an interactive Google Colab frontend.

The final model achieved a **test accuracy of 90.69%**, a **best validation accuracy of 95.00%**, a **multiclass ROC-AUC of 0.9821**, and a **macro PR-AUC of 0.9523**. These results are promising for an educational prototype and demonstrate that transfer learning with ResNet50 can effectively classify brain tumor MRI images into four categories. However, these results do not establish clinical reliability, and the system should be understood strictly as a research and learning exercise, requiring substantial further validation, testing, and expert medical review before any consideration of real-world application.

25. References

- 1 Nickparvar, M. Brain Tumor MRI Dataset. Kaggle.
<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>
- 2 PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>
- 3 Torchvision Documentation. <https://pytorch.org/vision/stable/index.html>
- 4 He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- 5 Pan, S. J., & Yang, Q. (2010). A Survey on Transfer Learning. IEEE Transactions on Knowledge and Data Engineering.
- 6 Scikit-learn Documentation - Model Evaluation: ROC-AUC and Precision-Recall metrics.
https://scikit-learn.org/stable/modules/model_evaluation.html
- 7 To be added - Medical AI ethics and validation reference(s).

26. Appendix

A.1 Environment Summary

Item	Value
Development platform	Google Colab
Deep learning framework	PyTorch / Torchvision
Model	ResNet50 (ImageNet pretrained, layer4 fine-tuned)
Image size	224 x 224
Batch size	32
Optimizer	Adam (lr = 1e-5)
Loss function	CrossEntropyLoss
Epochs	15
Train / Validation split	80% / 20% (random seed = 42)
Best validation accuracy	95.00%
Test accuracy	90.69%

A.2 Placeholders Requiring Future Completion

- Exact total number of dataset images (to be verified from actual loaded dataset output)
- Detailed classification report (class-wise precision, recall, F1-score)
- Multiclass ROC curve and precision-recall curve plots
- Example MRI prediction visualizations
- Class distribution chart
- Additional medical AI ethics and validation references